

Assignment No 14

Name : Shivam Ramakant Deshmane

Domain : AI & ML

College : T.B Girwalkar Polytechnic Ambajogai

Q1.

Code :

```
import numpy as np
```

```
arr = np.random.randint(10, 101, (5, 6))
```

```
print("Array:")
```

```
print(arr)
```

```
print("\nShape of the array:", arr.shape)
```

```
print("Total number of elements (size):", arr.size)
```

```
print("Data type (dtype):", arr.dtype)
```

Output :

Array:

```
[[ 53 100  28  52  29  63]
 [ 69  30  61  22  64  48]
 [ 50  88  11  69  73  22]
 [ 97  93  44  22  73  18]
 [ 88  46  20  64  98  56]]
```

Shape of the array: (5, 6)

Total number of elements (size): 30

Data type (dtype): int64

Q2.

Code :

```
import numpy as np
```

```
arr = np.random.randint(1, 51, 20)
```

```
print("Array:")
```

```
print(arr)
```

```
print("\nMinimum Value:", np.min(arr))
```

```
print("Index of Minimum Value:", np.argmin(arr))
```

```
print("\nMaximum Value:", np.max(arr))
```

```
print("Index of Maximum Value:", np.argmax(arr))
```

```
print("\nSum of Elements:", np.sum(arr))
```

```
print("Mean:", np.mean(arr))
```

```
print("Standard Deviation:", np.std(arr))
```

Output :

```
Array:
[39  3 45 21 34 27 36  8 41 42 16 16 32 15 13]

Minimum Value: 3
Index of Minimum Value: 1

Maximum Value: 48
Index of Maximum Value: 15

Sum of Elements: 577
Mean: 28.85
Standard Deviation: 12.846302970115566
```

Q3.

Code :

```
import numpy as np

matrix = np.random.randint(20, 81, size=(4, 5))

print("4x5 Random Matrix:")
print(matrix)

print("\nMinimum Value:", np.min(matrix))
print("Maximum Value:", np.max(matrix))
print("Sum of All Elements:", np.sum(matrix))
print("Mean:", np.mean(matrix))
print("Standard Deviation:", np.std(matrix))

print("\nRow-wise Sum:")
print(np.sum(matrix, axis=1))

print("\nColumn-wise Sum:")
print(np.sum(matrix, axis=0))
```

Output :

```
4x5 Random Matrix:
```

```
[[63 34 34 60 48]
 [76 48 40 50 63]
 [30 69 68 60 55]
 [69 37 42 25 39]]
```

```
Minimum Value: 25
```

```
Maximum Value: 76
```

```
Sum of All Elements: 1010
```

```
Mean: 50.5
```

```
Standard Deviation: 14.627029773675856
```

```
Row-wise Sum:
```

```
[239 277 282 212]
```

```
Column-wise Sum:
```

```
[238 188 184 195 205]
```

Q4.

Code :

```
import numpy as np
```

```
arr = np.arange(1, 25)
```

```
print("Original 1D Array:")
```

```
print(arr)
```

```
print("Shape:", arr.shape)
```

```
arr_4x6 = arr.reshape(4, 6)
```

```
print("\nReshaped Array (4, 6):")
```

```
print(arr_4x6)
```

```
print("Shape:", arr_4x6.shape)
```

```
arr_3x8 = arr.reshape(3, 8)
```

```
print("\nReshaped Array (3, 8):")
```

```
print(arr_3x8)
```

```
print("Shape:", arr_3x8.shape)
```

```
arr_2x3x4 = arr.reshape(2, 3, 4)
```

```
print("\nReshaped Array (2, 3, 4):")
```

```
print(arr_2x3x4)
```

```
print("Shape:", arr_2x3x4.shape)
```

Output :

Original 1D Array:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

Shape: (24,)

Reshaped Array (4, 6):

```
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
```

Shape: (4, 6)

Reshaped Array (3, 8):

```
[[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]
 [17 18 19 20 21 22 23 24]]
```

Shape: (3, 8)

Reshaped Array (2, 3, 4):

```
[[[ 1  2  3  4]
   [ 5  6  7  8]
   [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
```

Shape: (2, 3, 4)

Q5.

Code :

```
import numpy as np
```

```
arr = np.array([[10, 20, 30, 40],  
               [50, 60, 70, 80],  
               [90, 100, 110, 120]])
```

```
print("Original Array:")
```

```
print(arr)
```

```
print("\nFirst Row:")
```

```
print(arr[0])
```

```
print("\nLast Column:")
```

```
print(arr[:, -1])
```

```
print("\nCenter 2x2 Submatrix:")
```

```
print(arr[0:2, 1:3])
```

```
print("\nEven Numbers:")
```

```
print(arr[arr % 2 == 0])
```

Output :

```
Original Array:
```

```
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]
```

```
First Row:
```

```
[10 20 30 40]
```

```
Last Column:
```

```
[ 40  80 120]
```

```
Center 2x2 Submatrix:
```

```
[[20 30]
 [60 70]]
```

```
Even Numbers:
```

```
[ 10  20  30  40  50  60  70  80  90 100 110 120]
```

Q6.

Code :

```
import numpy as np

# Create a 5x5 matrix of random integers between 1 and 100
arr = np.random.randint(1, 101, size=(5, 5))

print("Original Array:")
print(arr)

# Print the diagonal elements
print("\nDiagonal Elements:")
print(np.diag(arr))

print("\nElements Greater Than 50:")
print(arr[arr > 50])

arr[arr < 30] = 0

print("\nModified Array (Elements < 30 replaced with 0):")
print(arr)
```

Output :

Original Array:

```
[[ 6  5 61 39 52]
 [97 34 31 75 79]
 [ 1 29 42 64 50]
 [30 26 59  4 87]
 [11 64 88 95 17]]
```

Diagonal Elements:

```
[ 6 34 42  4 17]
```

Elements Greater Than 50:

```
[61 52 97 75 79 64 59 87 64 88 95]
```

Modified Array (Elements < 30 replaced with 0)

```
[[ 0  0 61 39 52]
 [97 34 31 75 79]
 [ 0  0 42 64 50]
 [30  0 59  0 87]
 [ 0 64 88 95  0]]
```

Q7.

Code :

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40])
```

```
b = np.array([1, 2, 3, 4])
```

```
print("Array a:", a)
```

```
print("Array b:", b)
```

```
print("\nAddition:")
```

```
print(a + b)
```

```
print("\nSubtraction:")
```

```
print(a - b)
```

```
print("\nMultiplication:")
```

```
print(a * b)
```

```
print("\nDivision:")
```

```
print(a / b)
```

```
print("\nElement-wise Power (a ** b):")
```

```
print(a ** b)
```

Output :

```
Array a: [10 20 30 40]
```

```
Array b: [1 2 3 4]
```

```
Addition:
```

```
[11 22 33 44]
```

```
Subtraction:
```

```
[ 9 18 27 36]
```

```
Multiplication:
```

```
[ 10  40  90 160]
```

```
Division:
```

```
[10. 10. 10. 10.]
```

```
Element-wise Power (a ** b):
```

```
[      10      400    27000 2560000]
```

Q8 .

Code :

```
import numpy as np
```

```
A = np.array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
B = np.array([[9, 8, 7],  
              [6, 5, 4],  
              [3, 2, 1]])
```

```
elementwise = A * B
```

```
matrix_mul = A @ B    # or np.dot(A, B)
```

```
print("Matrix A:")
```

```
print(A)
```

```
print("\nMatrix B:")
```

```
print(B)
```

```
print("\nElement-wise Multiplication (A * B):")
```

```
print(elementwise)
```

```
print("\nMatrix Multiplication (A @ B):")
```

Output :

```
Array a: [10 20 30 40]
```

```
Array b: [1 2 3 4]
```

```
Addition:
```

```
[11 22 33 44]
```

```
Subtraction:
```

```
[ 9 18 27 36]
```

```
Multiplication:
```

```
[ 10  40  90 160]
```

```
Division:
```

```
[10. 10. 10. 10.]
```

```
Element-wise Power (a ** b):
```

```
[      10      400    27000 2560000]
```


Q9.

Code :

```
import numpy as np
```

```
matrix = np.random.randn(6, 6)
```

```
print("Original Matrix:\n", matrix)
```

```
print("\nShape:", matrix.shape)
```

```
print("Size:", matrix.size)
```

```
print("Data Type:", matrix.dtype)
```

```
max_index = np.unravel_index(np.argmax(matrix), matrix.shape)
```

```
min_index = np.unravel_index(np.argmin(matrix), matrix.shape)
```

```
print("\nMaximum Value:", matrix[max_index], "at index", max_index)
```

```
print("Minimum Value:", matrix[min_index], "at index", min_index)
```

```
print("\nTop-left 3x3 Submatrix:")
```

```
print(matrix[:3, :3])
```

```
matrix = np.abs(matrix)
```

```
print("\nModified Matrix (Negative values converted to Absolute):")
```

```
print(matrix)
```

```
print("\nMean of Modified Matrix:", np.mean(matrix))
```

Output :

```
Matrix A:
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
Matrix B:
```

```
[[9 8 7]
 [6 5 4]
 [3 2 1]]
```

```
Element-wise Multiplication (A * B):
```

```
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
```

```
Matrix Multiplication (A @ B):
```

```
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Q10.

Code :

```
import numpy as np

# Step 1: Generate a 2D array (10 students × 5 subjects)
marks = np.random.randint(30, 101, size=(10, 5))

# Print the generated marks
print("Student Marks (10 x 5):")
print(marks)

# Step 2: Calculate total marks and average for each student

total_marks = np.sum(marks, axis=1)
average_marks = np.mean(marks, axis=1)

print("\nTotal Marks of Each Student:")
print(total_marks)

print("\nAverage Marks of Each Student:")
print(average_marks)

# Step 3: Find student with highest and lowest total marks
```

```
# using argmax() and argmin()

highest_student = np.argmax(total_marks)
lowest_student = np.argmin(total_marks)

print("\nStudent with Highest Total Marks:")
print("Student Index:", highest_student)
print("Total Marks:", total_marks[highest_student])

print("\nStudent with Lowest Total Marks:")
print("Student Index:", lowest_student)
print("Total Marks:", )

# Step 4: Calculate overall class mean and standard deviation
class_mean = np.mean(marks)
class_std = np.std(marks)

print("\nOverall Class Mean:", class_mean)
print("Overall Class Standard Deviation:", class_std)

# Step 5: Reshape the array (demonstration)
# Here we reshape 10x5 into 5x10

reshaped_marks = marks.reshape(5, 10)

print("\nReshaped Array (5 x 10):")
print(reshaped_marks)
```

```
# Step 6: Extract marks of the top 3 students
top3_indices = np.argsort(total_marks)[-3:][::-1]

print("\nTop 3 Student Indices:", top3_indices)

print("\nMarks of Top 3 Students:")
print(marks[top3_indices])
```

Output :

Student Marks (10 x 5):

```
[[37 71 35 91 69]
 [38 49 99 67 43]
 [95 95 36 97 57]
 [36 63 59 38 55]
 [48 60 49 86 83]
 [74 60 90 59 59]
 [40 67 72 63 80]
 [73 56 93 49 49]
 [76 94 83 83 57]
 [96 47 59 95 46]]
```

Total Marks of Each Student:

```
[303 296 380 251 326 342 322 320 393 343]
```

Average Marks of Each Student:

```
[60.6 59.2 76.  50.2 65.2 68.4 64.4 64.  78.6
```

Student with Highest Total Marks:

Student Index: 8

Total Marks: 393

Student with Lowest Total Marks:

Student Index: 3

Total Marks: 251

Overall Class Mean: 65.52

Overall Class Standard Deviation: 19.600244896

Reshaped Array (5 x 10):

```
[[37 71 35 91 69 38 49 99 67 43]
 [95 95 36 97 57 36 63 59 38 55]
 [48 60 49 86 83 74 60 90 59 59]
 [40 67 72 63 80 73 56 93 49 49]
 [76 94 83 83 57 96 47 59 95 46]]
```

Top 3 Student Indices: [8 2 9]

Marks of Top 3 Students:

```
[[76 94 83 83 57]
 [95 95 36 97 57]
 [96 47 59 95 46]]
```